# Entrelíneas

## Fase 0 — Preparación

Repositorio, API, web, base de datos y tests

Proyecto de portfolio · Generado el 4 de octubre de 2026

# Índice


1. Fase 0 — Preparación
    - Resumen
    - Qué se construyó
    - Cómo está organizado el repositorio
    - La API por dentro
    - La web por dentro
    - Base de datos
    - Tests
    - Variables de entorno y secretos
    - Git y GitHub
    - Problemas encontrados y cómo se resolvieron
    - Cómo probar esta fase
    - Lo que sigue: Fase 1 — Base de datos y usuarios
2. Anexo — Prisma explicado
    - 1. Qué es Prisma y qué problema resuelve
    - 2. Las piezas y cómo se relacionan
    - 3. La conexión
    - 4. Escribir el esquema
    - 5. Generar el cliente
    - 6. Migraciones: cómo cambia la base sin perder datos
    - 7. Leer datos
    - 8. Escribir datos
    - 9. Dinero: el tipo Decimal
    - 10. Transacciones: todo o nada
    - 11. Errores de Prisma que vamos a manejar
    - 12. Dónde se usa Prisma dentro de la API
    - 13. Tests contra la base de prueba
    - 14. Datos de ejemplo (seed)
    - 15. Prisma Studio
    - 16. Resumen de comandos
    - 17. Glosario

# Fase 0 — Preparación

## Resumen

| | |
|---|---|
| **Objetivo** | Dejar armada la base técnica del proyecto: repositorio, API, web, base de datos y tests, todo conectado y funcionando |
| **Estado** | Completa |
| **Fecha** | 4 de octubre de 2026 |
| **Rama** | `feat/fase-0-preparacion` |
| **Resultado verificable** | `GET /api/salud/db` responde `conectada` pasando por web → API → MySQL; 6 tests en verde; `next build` sin errores |

Al terminar esta fase todavía no hay ninguna función de la librería (catálogo, pedidos, caja). Lo que hay es el **esqueleto**: cada pieza en su lugar, conectada con las demás y con las reglas del proyecto aplicadas desde la primera línea. Las fases siguientes van a agregar funciones sobre esta base, sin rehacerla.

## Qué se construyó

| Tarea de `task.md` | Cómo quedó |
|---|---|
| Decidir el acceso a datos | Prisma ORM 7.10 (ver el anexo "Prisma explicado" al final de este documento) |
| Repositorio con `apps/web`, `apps/api` y `docs/` | Monorepo con *npm workspaces*, en GitHub |
| API: Express, carpetas, errores, Zod, `.env.example` | Express 5 con capas, `AppError`, middleware central de errores y validación con Zod |
| Web: Next.js, Tailwind, fuentes, tokens | Next.js 16.3, Tailwind 4 con la paleta de `design.md`, Literata y Public Sans |
| Rewrite de `/api/*` hacia Express | En `apps/web/next.config.mjs` |
| MySQL local y conexión desde la API | Tres bases y un usuario propio con permisos acotados |
| Vitest y Supertest con base de prueba | 6 tests contra `entrelineas_test` |
| *(extra)* Actualizar a Node.js 24 | Requisito de Prisma 7 en un proyecto JavaScript |

## Cómo está organizado el repositorio

```
Entrelineas/
├── apps/
│   ├── api/                  ← Express + Prisma (puerto 4000)
│   │   ├── prisma/           ← schema.prisma y migraciones
│   │   ├── prisma7.config.ts ← configuración de la CLI de Prisma
│   │   ├── src/
│   │   │   ├── app.js        ← arma la aplicación Express
│   │   │   ├── server.js     ← la pone a escuchar en un puerto
│   │   │   ├── routes/  controllers/  services/  repositories/
│   │   │   ├── middlewares/  ← errores y validación
│   │   │   ├── errors/       ← AppError
│   │   │   ├── db/prisma.js  ← el único cliente de Prisma
│   │   │   └── generated/    ← cliente generado por Prisma (no va a Git)
│   │   ├── test/             ← tests de Vitest
│   │   └── .env              ← secretos locales (no va a Git)
│   └── web/                  ← Next.js (puerto 3000)
│       ├── app/              ← páginas (App Router)
│       ├── components/       ← componentes React (Logo.jsx)
│       └── next.config.mjs   ← rewrite de /api
├── docs/                     ← toda la documentación y el generador de PDF
├── package.json              ← raíz del monorepo
└── README.md
```

### Monorepo y *npm workspaces*

Un **monorepo** es un solo repositorio que contiene varios proyectos: acá, la API y la web. Las dos se versionan juntas, así un cambio que toca ambas (por ejemplo, un endpoint nuevo y la pantalla que lo usa) queda en un mismo commit.

Los **workspaces** de npm permiten manejar esos proyectos desde la raíz:

- Un solo `npm install` instala las dependencias de todos.
- Un solo `package-lock.json` fija las versiones exactas.
- Las dependencias compartidas se instalan una sola vez en `node_modules/` de la raíz, lo que ahorra espacio.
- Los scripts se ejecutan desde la raíz: `npm run dev:api`, `npm run dev:web`, `npm test`, `npm run docs:pdf`.

# La API por dentro

## El recorrido de un pedido HTTP

Cada pedido atraviesa siempre las mismas capas, y cada capa tiene una sola responsabilidad (regla de `rules.md`, sección 3). Ejemplo real con `GET /api/salud/db`:

```
GET /api/salud/db
  |
  ▼ app.js            express.json() lee el cuerpo; todo /api va al router
  ▼ routes/           salud-routes.js: "GET /db → obtenerSaludBaseDeDatos"
  ▼ controllers/      lee el pedido, llama al servicio, arma la respuesta { data }
  ▼ services/         lógica: si la base no responde, lanza AppError 503
  ▼ repositories/     única capa que habla con la base: prisma.$queryRaw`SELECT 1`
  ▼ MySQL
```

| Capa | Hace | No hace |
|---|---|---|
| **Route** | Asocia método y URL con un controller | Lógica de ningún tipo |
| **Controller** | Traduce HTTP ↔ JavaScript: lee `req`, escribe `res` | Consultar la base |
| **Service** | Reglas del negocio (stock, caja, pagos) | Saber de HTTP o de Prisma |
| **Repository** | Consultas a la base con Prisma | Tomar decisiones de negocio |

**Por qué tanta separación:** cuando en la Fase 5 haya que calcular el cierre de caja, esa cuenta va a vivir en un *service*. Se va a poder probar sin levantar un servidor (sin HTTP) y sin depender de cómo se escriben las consultas. Además, cada archivo va a ser corto y fácil de encontrar.

## Errores: `AppError` y el middleware central

Toda respuesta de la API tiene una de estas dos formas (regla de `rules.md`):

```
{ "data": { "estado": "ok" } }
{ "error": { "codigo": "NO_ENCONTRADO", "mensaje": "La ruta pedida no existe." } }
```

Para lograrlo:

1. Cuando algo sale mal por una regla del negocio, el código lanza un `AppError` con tres datos: estado HTTP, código y mensaje.

   ```
   throw new AppError(409, 'SIN_STOCK', 'No hay stock suficiente de El Aleph (quedan 1). Ajustá la can
   ```

   - El **código** (`SIN_STOCK`) es para el frontend: nunca cambia, y la web puede decidir qué hacer según el código.
   - El **mensaje** es para la persona: en español, con voseo, y dice cómo seguir.

2. Express 5 atrapa los errores, incluso los de funciones `async`, y los pasa al **middleware central** (`middlewares/errores.js`), que es el último de la cadena.
3. El middleware responde según el caso:
    - si es un `AppError`, devuelve su estado, código y mensaje;
    - si el JSON del cuerpo está mal formado, devuelve `400 JSON_INVALIDO`;
    - cualquier otro error es inesperado: lo escribe en la consola del servidor y al cliente le devuelve un `500` genérico, **sin detalles internos** ni *stack trace* (la lista de funciones donde ocurrió el error). Mostrar eso le daría pistas a un atacante.

Una ruta que no existe cae en `rutaNoEncontrada`, que genera un `AppError(404)`.

### Validación con Zod

**Zod** es una librería para describir cómo tienen que ser los datos y verificarlos. El middleware `validar(esquema)` se pone delante del controller:

```
const esquemaPedido = z.object({
  nombre: z.string().min(1),
  telefono: z.string().min(8),
  email: z.email().optional(),
});

router.post('/pedidos', validar(esquemaPedido), crearPedido);
```

- Si los datos cumplen el esquema, quedan limpios en `req.validado.body` y el controller trabaja solo con eso.
- Si no, el pedido se corta con `400 DATOS_INVALIDOS` y un mensaje que dice qué campo falló (`telefono: …`).

Así ningún controller recibe datos sin verificar (regla de `rules.md`: "toda entrada se valida con Zod").

### `app.js` y `server.js`: por qué son dos archivos

- `app.js` **arma** la aplicación (middlewares, rutas, errores) y la devuelve, sin ponerla a escuchar.
- `server.js` la **pone a escuchar** en el puerto y maneja el apagado ordenado (cierra las conexiones con MySQL al recibir `Ctrl+C`).

La separación es para los tests: Supertest recibe la aplicación de `app.js` y le hace pedidos en memoria, sin ocupar ningún puerto.

## La web por dentro

### Next.js y el App Router

Next.js es un framework de React que genera las páginas **en el servidor**. Para el catálogo es clave: cuando alguien comparte el link de un libro por WhatsApp, WhatsApp lee el HTML y muestra la

portada y el precio. Eso solo funciona si el HTML ya viene armado, sin esperar a que corra JavaScript en el navegador.

En el **App Router**, cada carpeta dentro de `app/` es una ruta y su `page.js` es la página:

| Archivo | URL |
|---|---|
| `app/page.js` | `/` |
| `app/libros/page.js` | `/libros` (Fase 2) |
| `app/libros/[slug]/page.js` | `/libros/el-aleph-jorge-luis-borges` (Fase 2) |

`app/layout.js` envuelve a todas las páginas: ahí se cargan las fuentes, el idioma (`lang="es-AR"`) y el título por defecto.

## El rewrite: web y API en el mismo dominio

```
// apps/web/next.config.mjs
async rewrites() {
  return [{ source: '/api/:path*', destination: `${apiUrl}/api/:path*` }];
}
```

Cuando el navegador pide `localhost:3000/api/salud`, Next.js reenvía el pedido a Express (`localhost:4000/api/salud`) y devuelve la respuesta como si fuera propia. El navegador nunca se entera de que existe otro servidor.

**Por qué importa:** en la Fase 1 el login va a guardar la sesión en una *cookie*. Si la web y la API estuvieran en dominios distintos, el navegador trataría esa cookie como "de terceros" y habría que configurar CORS (las reglas que permiten pedidos entre dominios), lo que es complejo y propenso a errores de seguridad. Con el rewrite, todo está en el mismo dominio y la cookie viaja sola.

`API_URL` sale de `.env.local`. En la demo apunta a la API publicada en Railway o Render.

## Tailwind con los tokens de diseño

Tailwind es una forma de dar estilos con clases cortas directamente en el HTML (`bg-papel`, `text-grafito`, `mt-6`). En Tailwind 4, la paleta del proyecto se define en `app/globals.css` con `@theme`:

```
@theme {
  --color-papel: #F8F8F5;
  --color-tela: #1F4A3D;
  --color-ocre: #B07D24;
  /* … */
}
```

Cada `--color-nombre` crea las clases `bg-nombre`, `text-nombre` y `border-nombre`. Como `rules.md` prohíbe colores sueltos fuera de la paleta, siempre se usan estas clases y nunca un `#hex` directamente en un

componente.

También quedaron aplicadas desde el inicio dos reglas de accesibilidad:

- **Foco visible:** contorno verde tela de 2 px en todo elemento enfocado con el teclado.
- **`prefers-reduced-motion`:** si la persona pidió menos animaciones en su sistema, se desactivan todas.

### Fuentes con `next/font`

Literata y Public Sans se cargan con `next/font/google`. Next.js descarga los archivos en el momento del *build* y los sirve desde el mismo sitio. No hay pedidos a Google cuando alguien visita la página (mejor privacidad y velocidad), y se evita el salto visual cuando la fuente termina de cargar.

## Base de datos

| Base | Para qué | Quién la usa |
|---|---|---|
| `entrelineas` | Desarrollo | La API con `npm run dev:api` |
| `entrelineas_shadow` | Base sombra de Prisma | Solo `prisma migrate dev` (se borra sola) |
| `entrelineas_test` | Tests | Vitest |

El usuario `entrelineas` tiene permisos **solo** sobre esas tres bases. El detalle de la conexión, la configuración y los conceptos de Prisma está en el anexo al final de este documento.

## Tests

**Vitest** ejecuta los tests y **Supertest** le hace pedidos HTTP a la aplicación Express sin levantar un servidor.

```
it('una ruta inexistente devuelve 404 con la forma { error }', async () => {
  const res = await request(app).get('/api/no-existe');
  expect(res.status).toBe(404);
  expect(res.body).toEqual({ error: { codigo: 'NO_ENCONTRADO', mensaje: 'La ruta pedida no existe.' } });
});
```

Cada test se lee como una frase: "hacé este pedido, **esperá** (`expect`) este resultado".

| Archivo | Qué verifica |
|---|---|
| `test/salud.test.js` | La API responde con `{ data }`; se conecta a la base **de prueba** (y no a la de desarrollo); 404 con `{ error }`; un JSON roto devuelve 400 sin detalles internos |
| `test/validar.test.js` | `validar` deja pasar datos correctos y rechaza los incorrectos con `AppError 400` |

`test/setup.js` cambia `DATABASE_URL` por `TEST_DATABASE_URL` antes de cada archivo. Así los tests nunca tocan los datos de desarrollo. Se ejecutan con `npm test` desde la raíz.

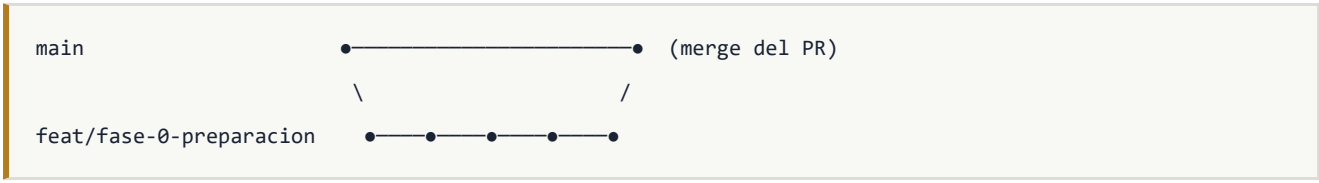
## Variables de entorno y secretos

Los datos sensibles (contraseñas, claves de Mercado Pago) **nunca van al repositorio**:

| Archivo | ¿Va a Git? | Contenido |
|---|---|---|
| `apps/api/.env` | No | Valores reales: la contraseña de MySQL |
| `apps/api/.env.example` | Sí | Las mismas variables con valores de ejemplo |
| `apps/web/.env.local` | No | `API_URL` real |
| `apps/web/.env.example` | Sí | Plantilla |

Los `.env.example` documentan qué variables hay que completar. Quien clone el repositorio los copia como `.env` y pone sus valores. Antes de cada commit se verifica que ningún archivo con secretos esté por subirse.

## Git y GitHub




- `main` tiene siempre una versión estable. Es la que Vercel va a publicar.
- Cada fase se trabaja en su rama (`feat/fase-0-preparacion`, `feat/fase-1-base-de-datos`, ...).
- Al terminar la fase se abre un **pull request** (PR): la propuesta de sumar la rama a `main`, donde se revisan los cambios antes de aceptarlos.
- Los commits van en español, en presente (`agrega cierre de caja con arqueo`), según `rules.md`.

Repositorio: https://github.com/CyntiaCarrizo/Entrelineas

## Problemas encontrados y cómo se resolvieron

| Problema | Causa | Solución |
|---|---|---|
| `npm install` fallaba con `ENOSPC` | Disco C: con 0,25 GB libres | Se vació la papelera y la caché de npm. Después de cada instalación se vuelve a limpiar la caché |
| El cliente de Prisma 7 no corría en Node 20 | Prisma 7 genera el cliente en TypeScript | Actualización a Node 24 LTS, que ejecuta TypeScript directamente. Además, Node 20 ya no tenía soporte |
| `npm` no ejecutaba los scripts de Prisma | npm 11 bloquea por seguridad los scripts de instalación | Se revisó qué hacía cada script y se aprobaron con `npm approve-scripts` |

| Problema | Causa | Solución |
|---|---|---|
| Prisma daba `pool timeout` | MySQL 9 autentica con `caching_sha2_password` y el driver no pedía la clave pública | `?allowPublicKeyRetrieval=true` en la URL de conexión |
| Riesgo de borrar bases de otros proyectos | El permiso global `CREATE, DROP` que Prisma pide para la base sombra | Base sombra creada a mano y permisos solo sobre las bases del proyecto |
| El puerto 4000 respondía con otra API | Otro proyecto local (bonhomia) usa el mismo puerto | Pruebas en los puertos 4100 y 3100. Al liberar el puerto se cortó un momento el servidor de bonhomia, que se reinició solo |

## Cómo probar esta fase

```
npm install
npm run db:check -w apps/api     # "Conexión correcta: base entrelineas, MySQL 9.3.0"
npm test                         # 6 tests en verde
npm run dev:api                  # API en http://localhost:4000
npm run dev:web                  # Web en http://localhost:3000
```

Con las dos en marcha:

- `http://localhost:3000` muestra la página provisoria con el logo.
- `http://localhost:3000/api/salud/db` devuelve `{"data":{"baseDeDatos":"conectada"}}`, pasando por la web, la API y MySQL.

## Lo que sigue: Fase 1 — Base de datos y usuarios

- Escribir en `schema.prisma` todos los modelos de `arquitectura.md` y crear la primera migración.
- Cargar los datos de ejemplo (*seed*): 2 usuarios, 8 géneros y 60 libros.
- Login y logout con JWT en una cookie `httpOnly`, y roles `admin` y `vendedor`.
- Pantalla de login y layout del panel de administración.

# Anexo — Prisma explicado

Esta guía explica Prisma desde cero y con ejemplos de este proyecto: cómo se conecta a MySQL, cómo se escribe el esquema, cómo se consultan y modifican datos, y los conceptos que hay que tener claros para no romper el dinero ni el stock. Está pensada para leerse de corrido la primera vez y después usarse como consulta.

Versión usada: **Prisma ORM 7.10** (la última estable al empezar el proyecto) con MySQL 9.3 y Node.js 24.

## 1. Qué es Prisma y qué problema resuelve

Una aplicación en Node.js no "habla" MySQL directamente: necesita armar consultas SQL como texto, mandarlas a la base y convertir las filas que vuelven en objetos de JavaScript. Hacerlo a mano funciona, pero es fácil equivocarse en el nombre de una columna, olvidarse de un campo o escribir una consulta vulnerable.

**Prisma es un ORM** (*Object-Relational Mapper*, mapeador objeto-relacional): una herramienta que hace de traductor entre las tablas de la base y los objetos de JavaScript. En vez de escribir:

```
SELECT id, titulo, precio FROM libros WHERE activo = 1 ORDER BY titulo LIMIT 20;
```

se escribe:

```
const libros = await prisma.libro.findMany({
  where: { activo: true },
  select: { id: true, titulo: true, precio: true },
  orderBy: { titulo: 'asc' },
  take: 20,
});
```

y Prisma genera el SQL, lo ejecuta y devuelve un arreglo de objetos.

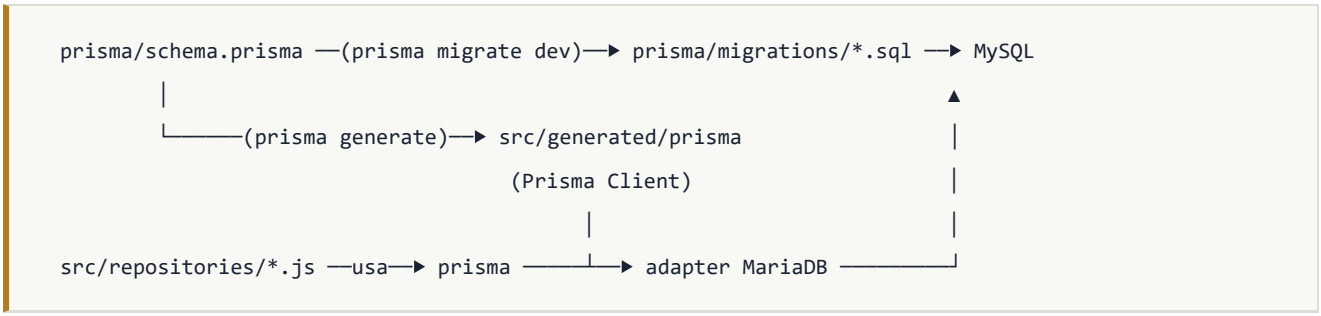
### Por qué lo elegimos para Entrelíneas

| Ventaja | Qué significa en el proyecto |
|---|---|
| Un solo archivo describe toda la base | `schema.prisma` es la fuente de verdad del modelo de datos de `arquitectura.md` |
| Migraciones incluidas | Cada cambio en la base queda versionado en Git como un archivo SQL |
| Consultas parametrizadas siempre | Protección contra inyección SQL sin pensarlo |
| Transacciones simples | Clave para la caja, los pagos y el stock |

| Ventaja | Qué significa en el proyecto |
|---|---|
| Prisma Studio | Una interfaz web para ver y editar datos durante el desarrollo |

**Qué no hace Prisma por nosotros:** no decide las reglas del negocio. Que el stock nunca quede negativo o que un pago no se registre dos veces sigue siendo responsabilidad nuestra (ver secciones 9 y 10).

## 2. Las piezas y cómo se relacionan




| Pieza | Qué es | Dónde vive |
|---|---|---|
| **Esquema** | Archivo que describe tablas, columnas y relaciones con el lenguaje de Prisma | `apps/api/prisma/schema.prisma` |
| **Configuración** | Le dice a la CLI de Prisma dónde está el esquema, dónde guardar las migraciones y a qué base conectarse | `apps/api/prisma7.config.ts` |
| **CLI** ( `prisma` ) | Programa de línea de comandos: crea migraciones, genera el cliente, abre Studio. Solo se usa en desarrollo y en el deploy | Dependencia de desarrollo |
| **Migraciones** | Archivos `.sql` que Prisma escribe a partir de los cambios del esquema | `apps/api/prisma/migrations/` |
| **Prisma Client** | Código generado a medida del esquema: `prisma.libro` , `prisma.venta` , etc. | `apps/api/src/generated/prisma/` (no se sube a Git; se regenera) |
| **Driver adapter** | El conector real con MySQL. Prisma 7 lo exige: el cliente ya no trae su propio motor, usa un driver de JavaScript | Paquete `@prisma/adapter-mariadb` |

El adapter se llama "MariaDB" porque usa el driver `mariadb` de npm, que habla el mismo protocolo que MySQL. Es el adapter oficial de Prisma para ambas bases.

## 3. La conexión

### 3.1 La cadena de conexión

MySQL necesita cinco datos para aceptar una conexión: servidor, puerto, usuario, contraseña y base. Se escriben juntos en una URL:

```
mysql://USUARIO:CONTRASEÑA@SERVIDOR:PUERTO/BASE
mysql://entrelineas:clave-segura@localhost:3306/entrelineas?allowPublicKeyRetrieval=true
```

Lo que va después del `?` son **opciones** de la conexión. `allowPublicKeyRetrieval=true` hace falta con MySQL 8 y 9: el método de autenticación por defecto (`caching_sha2_password`) necesita la clave pública del servidor, y sin esta opción el driver no la pide y la conexión falla (en Prisma aparece como `pool timeout`). En local es seguro, porque la conexión no sale de la máquina. En la demo se va a usar una conexión con SSL.

Si la contraseña tiene caracteres especiales (`@`, `:`, `/`, `#`), hay que codificarlos (`@` se escribe `%40`), porque si no rompen la URL.

Esa URL va en el archivo `.env` de `apps/api`, que **nunca se sube a Git** (está en `.gitignore`). En el repositorio solo va `.env.example`, con la misma variable y un valor de ejemplo:

```
# apps/api/.env.example
DATABASE_URL="mysql://entrelineas:contraseña@localhost:3306/entrelineas?allowPublicKeyRetrieval=true"
SHADOW_DATABASE_URL="mysql://entrelineas:contraseña@localhost:3306/entrelineas_shadow?allowPublicKeyRetrie
TEST_DATABASE_URL="mysql://entrelineas:contraseña@localhost:3306/entrelineas_test?allowPublicKeyRetrieval=
```

Son tres bases: la de desarrollo, la **sombra**, que Prisma usa para las migraciones (sección 6.3), y la de **tests**, que se puede vaciar sin miedo.

En el deploy (Railway o Render), la misma variable se carga en el panel del proveedor.

### 3.2 Usuario propio para la aplicación

No conviene que la aplicación use `root`. Se crea un usuario solo para el proyecto, con permisos **únicamente sobre sus tres bases**:

```
CREATE DATABASE entrelineas        CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE DATABASE entrelineas_shadow CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE DATABASE entrelineas_test   CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'entrelineas'@'localhost' IDENTIFIED BY 'clave-segura';
GRANT ALL PRIVILEGES ON entrelineas.*        TO 'entrelineas'@'localhost';
GRANT ALL PRIVILEGES ON entrelineas_shadow.* TO 'entrelineas'@'localhost';
GRANT ALL PRIVILEGES ON entrelineas_test.*   TO 'entrelineas'@'localhost';
```

Esto sigue el **principio de mínimo privilegio**: si la aplicación tuviera un error o una vulnerabilidad, no podría tocar otras bases del mismo servidor. Por eso no se le da el permiso global `CREATE, DROP ON *.*`, que Prisma necesitaría para crear la base sombra por su cuenta. En cambio, la creamos nosotros y se la indicamos en la configuración.

`utf8mb4` permite guardar cualquier carácter (tildes, eñes, emojis en un nombre de cliente). Es importante para una librería en español.

### 3.3 La configuración de la CLI

```
// apps/api/prisma7.config.ts
import 'dotenv/config';
import { defineConfig } from 'prisma/config';

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: { path: 'prisma/migrations' },
  datasource: {
    url: process.env.DATABASE_URL,
    shadowDatabaseUrl: process.env.SHADOW_DATABASE_URL,
  },
});
```

- `import 'dotenv/config'` lee el archivo `.env` y carga sus valores en `process.env`.
- Este archivo lo usa **solo la CLI** (para migrar, generar, abrir Studio). La aplicación en ejecución se conecta por su cuenta (siguiente punto).
- A partir de Prisma 7.10 el archivo se llama `prisma7.config.ts` para no confundirse con el formato de Prisma 8.
- Es el único archivo TypeScript que escribimos en el proyecto. Prisma lo lee por su cuenta, así que no hace falta saber TypeScript: es solo un objeto de configuración.

### 3.4 Un solo cliente para toda la API

Crear un `PrismaClient` abre un **pool de conexiones**: un grupo de conexiones a MySQL que se reutilizan entre pedidos HTTP, porque abrir una conexión nueva cada vez es lento. Por eso se crea **una sola vez** y se importa desde donde haga falta:

```
// apps/api/src/db/prisma.js
import { PrismaMariaDb } from '@prisma/adapter-mariadb';
import { PrismaClient } from '../generated/prisma/client.ts';

const adapter = new PrismaMariaDb(process.env.DATABASE_URL);

export const prisma = new PrismaClient({ adapter });
```

Dos detalles de este archivo:

- El cliente generado está en TypeScript (`client.ts`). Node.js 24 lo ejecuta directamente: le quita los tipos al cargarlo (*type stripping*), sin compilar nada. Por eso el proyecto exige Node 24; con

Node 20 este import falla.

- `PrismaMariaDb` recibe la misma `DATABASE_URL` que usa la CLI, así hay una sola fuente de verdad para la conexión.

Al apagar el servidor se cierra el pool con `await prisma.$disconnect()`, para no dejar conexiones colgadas. En el proyecto lo hace `src/server.js` cuando recibe `Ctrl+C`.

### 3.5 Probar que conecta

```
await prisma.$queryRaw`SELECT 1`;
```

Si la URL, el usuario o la contraseña están mal, esta línea falla con un error que lo dice. En el proyecto lo hace el script `npm run db:check`.

## 4. Escribir el esquema

El esquema usa un lenguaje propio de Prisma, simple y declarativo. Tiene tres tipos de bloques: `generator` (qué cliente generar), `datasource` (qué base) y `model` (cada tabla).

```
// apps/api/prisma/schema.prisma
generator client {
  provider = "prisma-client"
  output   = "../src/generated/prisma"
}

datasource db {
  provider = "mysql"
}
```

### 4.1 Un modelo = una tabla

Ejemplo real del proyecto, la tabla `generos`:

```
model Genero {
  id     Int     @id @default(autoincrement())
  nombre String  @db.VarChar(80)
  slug   String  @unique @db.VarChar(80)
  libros Libro[]

  @@map("generos")
}
```

Cómo se lee cada línea: **nombre del campo**, **tipo** y **atributos** (lo que empieza con `@`).

| Escritura | Significado |
|---|---|
| `Int`, `String`, `Boolean`, `DateTime`, `Decimal`, `Json` | Tipos de Prisma. Cada uno se traduce a un tipo de MySQL |
| `String?` | El `?` significa que acepta `NULL` (opcional) |
| `@id` | Clave primaria |
| `@default(autoincrement())` | MySQL asigna el número solo (`AUTO_INCREMENT`) |
| `@default(now())` | Toma la fecha y hora del momento de crear la fila |
| `@updatedAt` | Prisma la actualiza en cada modificación |
| `@unique` | No puede haber dos filas con el mismo valor |
| `@db.VarChar(80)`, `@db.Decimal(12, 2)`, `@db.Text` | Tipo exacto de MySQL, cuando el de Prisma no alcanza |
| `@map("columna")` / `@@map("tabla")` | Nombre real en la base (ver 4.2) |
| `@@unique([a, b])` | Unicidad combinada de dos columnas |
| `@@index([a])` | Índice para acelerar búsquedas |

Los atributos con una `@` se aplican a un campo; los de dos `@@`, a todo el modelo.

## 4.2 Nombres: snake_case en la base, camelCase en JavaScript

`rules.md` pide tablas y columnas en `snake_case` (`creado_en`), y en JavaScript se usa `camelCase` (`creadoEn`). Prisma resuelve las dos cosas con `@map` y `@@map`: el modelo se llama `Libro` y la tabla `libros`; el campo es `creadoEn` y la columna `creado_en`.

```
model Libro {
  id               Int      @id @default(autoincrement())
  isbn             String?  @unique @db.VarChar(13)
  codigoInterno    String?  @unique @map("codigo_interno") @db.VarChar(40)
  slug             String   @unique @db.VarChar(200)
  titulo           String   @db.VarChar(255)
  autor            String   @db.VarChar(255)
  editorial        String   @db.VarChar(150)
  generoId         Int      @map("genero_id")
  genero           Genero   @relation(fields: [generoId], references: [id])
  sinopsis         String?  @db.Text
  paginas          Int?
  precio           Decimal  @db.Decimal(12, 2)
  stock            Int      @default(0)
  portadaPublicId  String?  @map("portada_public_id") @db.VarChar(255)
  activo           Boolean  @default(true)
  creadoEn         DateTime @default(now()) @map("creado_en")
  actualizadoEn    DateTime @updatedAt @map("actualizado_en")

  @@map("libros")
}
```

Notar que `isbn` es `String` y no un número: así lo pide `arquitectura.md`. Un ISBN puede empezar con cero y no se hacen cuentas con él.

### 4.3 Relaciones

Una relación conecta dos modelos. En Entrelíneas la más importante es **una venta tiene muchos ítems**:

```
model Venta {
  id     Int         @id @default(autoincrement())
  codigo String      @unique @db.VarChar(12)
  total  Decimal     @db.Decimal(12, 2)
  items  VentaItem[]               // lado "muchos": no crea columna
  pagos  Pago[]
  // …resto de los campos

  @@map("ventas")
}

model VentaItem {
  id             Int     @id @default(autoincrement())
  ventaId        Int     @map("venta_id")
  venta          Venta   @relation(fields: [ventaId], references: [id])
  libroId        Int     @map("libro_id")
  libro          Libro   @relation(fields: [libroId], references: [id])
  titulo         String  @db.VarChar(255)       // copia, ver arquitectura.md
  precioUnitario Decimal @map("precio_unitario") @db.Decimal(12, 2)
  cantidad       Int
  subtotal       Decimal @db.Decimal(12, 2)

  @@map("venta_items")
}
```

- `ventaId` es la **columna real** (la clave foránea `venta_id`).
- `venta` es un **campo de relación**: no existe en la base, le dice a Prisma cómo navegar de un ítem a su venta.
- `items VentaItem[]` es el lado inverso: permite pedir una venta "con sus ítems".
- `@relation(fields: [ventaId], references: [id])` significa "`venta_id` apunta al `id` de `ventas`". En MySQL se convierte en un `FOREIGN KEY`.

Cuando dos relaciones apuntan al mismo modelo (en `cajas`, `abierta_por` y `cerrada_por` apuntan a `usuarios`), cada una lleva un nombre para distinguirlas: `@relation("CajaAbiertaPor", …)`.

## 4.4 Enums

Los estados con valores fijos se declaran como `enum`, que en MySQL se convierte en una columna `ENUM`:

```
enum MetodoPago {
  efectivo
  transferencia
  mercadopago
  debito
  credito
}

model Pago {
  id         Int        @id @default(autoincrement())
  metodo     MetodoPago
  referencia String?    @db.VarChar(100)
  // …

  @@unique([metodo, referencia])   // evita registrar dos veces el mismo pago
  @@map("pagos")
}
```

Si el código intenta guardar `metodo: 'cheque'`, Prisma lo rechaza antes de llegar a la base.

## 5. Generar el cliente

Cada vez que cambia el esquema, el cliente se tiene que regenerar para que refleje los modelos nuevos:

```
npx prisma generate
```

Esto escribe el código en `src/generated/prisma/`. Ese código no se edita a mano ni se sube a Git: se regenera en cada instalación y en cada deploy. `prisma migrate dev` también lo regenera solo.

## 6. Migraciones: cómo cambia la base sin perder datos

### 6.1 Qué es una migración

Es un archivo SQL con los cambios necesarios para pasar la base de un estado al siguiente: crear una tabla, agregar una columna, crear un índice. Las migraciones se aplican **en orden** y quedan en Git, así cualquier persona (o el servidor de la demo) puede reconstruir la misma base desde cero.

### 6.2 El ciclo de trabajo

1. Editar `schema.prisma` (por ejemplo, agregar el campo `paginas` a `Libro`).
2. Ejecutar:

```
npx prisma migrate dev --name agrega-paginas-a-libros
```

3. Prisma compara el esquema con la base, escribe `prisma/migrations/20261004_agrega-paginas-a-libros/migration.sql`, lo aplica y regenera el cliente.
4. Revisar el SQL generado y subirlo a Git junto con el cambio del esquema.

La tabla `_prisma_migrations` (la crea Prisma en la base) guarda qué migraciones ya se aplicaron.

### 6.3 La base "sombra"

`migrate dev` usa una base temporal, la **base sombra** (*shadow database*). La vacía, aplica ahí todas las migraciones desde cero y compara el resultado con el esquema para detectar diferencias o migraciones editadas a mano. En el proyecto es `entrelineas_shadow` (sección 3.2). **Nunca guardes datos ahí**, porque Prisma la borra en cada `migrate dev`. Esta base no se usa en producción.

### 6.4 Los tres comandos que importan

| Comando | Cuándo | Qué hace |
|---|---|---|
| `prisma migrate dev` | Desarrollo | Crea la migración nueva, la aplica y regenera el cliente |
| `prisma migrate deploy` | Deploy (Railway/Render) | Aplica las migraciones pendientes. Nunca crea nuevas ni borra datos |
| `prisma migrate reset` | Solo desarrollo | **Borra toda la base** y la rehace desde las migraciones (el seed se corre después, aparte) |

**Reglas:**

- Nunca editar una migración que ya se aplicó en otro entorno. Si hay un error, se hace otra migración que lo corrige.
- Si un cambio puede perder datos (borrar una columna, achicar un `VARCHAR`), Prisma avisa y pide confirmación. Leer el aviso.

## 7. Leer datos

Todas las operaciones se escriben con la forma `prisma.<modelo>.<operación>({ … })` y devuelven una promesa (por eso llevan `await`).

### 7.1 Buscar uno

```
// Un libro por slug, para la página /libros/[slug]
const libro = await prisma.libro.findUnique({
  where: { slug: 'el-aleph-jorge-luis-borges' },
});
// Devuelve el objeto, o null si no existe
```

`findUnique` solo acepta campos `@id` o `@unique`. Para cualquier otro filtro se usa `findFirst`.

### 7.2 Buscar muchos, con filtros y paginación

Así se resuelve el catálogo de `/libros?genero=novela&q=borges&pagina=2`:

```
const POR_PAGINA = 20;

const where = {
  activo: true,
  genero: { slug: 'novela' },             // filtro a través de la relación
  OR: [
    { titulo: { contains: 'borges' } },
    { autor: { contains: 'borges' } },
    { isbn: 'borges' },
  ],
};

const [libros, total] = await Promise.all([
  prisma.libro.findMany({
    where,
    orderBy: { titulo: 'asc' },
    skip: (2 - 1) * POR_PAGINA,           // salteá la página 1
    take: POR_PAGINA,                     // traé 20
  }),
  prisma.libro.count({ where }),
]);
```

Con la intercalación (*collation*) `utf8mb4_unicode_ci`, `contains` no distingue mayúsculas ni tildes: buscar "borges" encuentra "Borges".

### 7.3 Elegir qué columnas salen: `select`

`rules.md` exige que los endpoints públicos nunca devuelvan datos internos. Con `select` se piden solo los campos necesarios:

```
const libro = await prisma.libro.findUnique({
  where: { slug },
  select: {
    titulo: true, autor: true, precio: true, stock: true, sinopsis: true,
    genero: { select: { nombre: true, slug: true } },
  },
});
```

Todo lo que no está en `select` no sale de la base. Es la forma más segura de armar respuestas públicas.

### 7.4 Traer relaciones: `include`

```
// Una venta con sus ítems y pagos, para el admin
const venta = await prisma.venta.findUnique({
  where: { id: 15 },
  include: { items: true, pagos: true },
});
```

`include` agrega relaciones completas; `select` elige campo por campo. No se pueden usar los dos en el mismo nivel.

## 8. Escribir datos

```
// Crear
const genero = await prisma.genero.create({
  data: { nombre: 'Novela', slug: 'novela' },
});

// Modificar
await prisma.libro.update({
  where: { id: 7 },
  data: { precio: '18500.00' },
});

// Modificar con operaciones atómicas
await prisma.libro.update({
  where: { id: 7 },
  data: { stock: { decrement: 1 } },     // stock = stock - 1, en la base
});

// Crear una venta con sus ítems en una sola operación (escritura anidada)
await prisma.venta.create({
  data: {
    codigo: 'ENT-4821',
    total: '37000.00',
    items: {
      create: [
        { libroId: 7, titulo: 'El Aleph', precioUnitario: '18500.00', cantidad: 1, subtotal: '18500.00' },
        { libroId: 9, titulo: 'Ficciones', precioUnitario: '18500.00', cantidad: 1, subtotal: '18500.00' }
      ],
    },
  },
});
```

Sobre borrar: existe `delete`, pero en `ventas`, `pagos`, `cajas`, `movimientos_caja` y `facturas` **no se usa nunca**. Se anula con un `update` que completa `anuladoPor`, `anuladoEn` y `motivoAnulacion` (`rules.md`, sección 4).

## 9. Dinero: el tipo Decimal

Los montos están en la base como `DECIMAL(12,2)`, que guarda el valor exacto. En JavaScript, en cambio, `0.1 + 0.2` da `0.30000000000000004`, porque los números de JS (*float*) no son exactos.

Prisma devuelve los `Decimal` como un objeto especial (de la librería Decimal.js), no como un número común. La regla del proyecto es operar en **centavos enteros**:

```
// De la base a centavos
const centavos = Math.round(Number(libro.precio) * 100);   // 18500.00 → 1850000

// Cálculos siempre con enteros
const subtotal = centavos * cantidad;

// De centavos a la base: string con dos decimales
const paraGuardar = (subtotal / 100).toFixed(2);           // "37000.00"
```

Estas conversiones viven en un solo módulo (`src/lib/dinero.js`) para no repetirlas. Al guardar se pasa un string (`'37000.00'`), que Prisma convierte a `DECIMAL` sin pérdida.

# 10. Transacciones: todo o nada

## 10.1 Qué es una transacción

Una **transacción** agrupa varias operaciones para que se apliquen todas juntas o ninguna. Si al confirmar un pedido se descuenta el stock del primer libro y falla el segundo, la base no puede quedar a medias: se deshace todo (*rollback*). Es la propiedad que garantiza que la caja y el stock no se desordenen.

## 10.2 Transacción interactiva

Prisma ejecuta una función con un cliente especial, `tx`. Todo lo que se hace con `tx` va en la misma transacción. Si la función lanza un error, Prisma deshace todo:

```
// Confirmar un pedido: descuenta el stock de cada libro o no descuenta nada
await prisma.$transaction(async (tx) => {
  for (const item of venta.items) {
    const { count } = await tx.libro.updateMany({
      where: { id: item.libroId, stock: { gte: item.cantidad } },   // solo si alcanza
      data: { stock: { decrement: item.cantidad } },
    });
    if (count === 0) {
      throw new AppError(409, 'SIN_STOCK', `No hay stock suficiente de ${item.titulo}.`);
    }
    await tx.movimientoStock.create({
      data: { libroId: item.libroId, tipo: 'venta', cantidad: -item.cantidad, ventaId: venta.id },
    });
  }
  await tx.venta.update({
    where: { id: venta.id },
    data: { estado: 'confirmada', stockDescontado: true, confirmadaEn: new Date() },
  });
});
```

**Por qué `updateMany` con condición y no "leer, verificar y restar":** si la web y el mostrador venden el último ejemplar al mismo tiempo, los dos podrían leer `stock = 1`, los dos pasarían la verificación y el stock quedaría en `-1`. Con `updateMany` la condición `stock >= cantidad` y la resta ocurren en **una sola instrucción SQL**: MySQL bloquea la fila, así que la segunda venta encuentra `stock = 0`, `count` vale `0` y se rechaza. Es la regla "el stock nunca queda negativo" de `rules.md`.

Importante: dentro de la función se usa siempre `tx`, nunca `prisma`. Lo que se haga con `prisma` queda fuera de la transacción.

## 11. Errores de Prisma que vamos a manejar

Cuando falla una operación, Prisma lanza un error con un código:

| Código | Qué pasó | Uso en el proyecto |
|---|---|---|
| `P2002` | Se violó un `@unique` | Transferencia con número de operación repetido, webhook duplicado, ISBN repetido |
| `P2025` | No existe el registro que se quería modificar | Devolver 404 |
| `P2003` | Clave foránea inválida | Un `libroId` que no existe |

El webhook de Mercado Pago es **idempotente** gracias a `@@unique([metodo, referencia])`. Si llega el mismo aviso dos veces, el segundo `create` falla con `P2002` y se ignora:

```
try {
  await tx.pago.create({ data: { metodo: 'mercadopago', referencia: String(pagoMp.id), /* … */ } });
} catch (error) {
  if (error.code === 'P2002') return;   // ya estaba registrado: no hacer nada
  throw error;
}
```

El middleware central de errores traduce estos códigos a `AppError` con mensajes en español, y nunca devuelve el error original al cliente.

## 12. Dónde se usa Prisma dentro de la API

Según `rules.md`, **solo la capa `repositories/` importa `prisma`**. Los servicios piden datos a los repositorios y no saben que existe Prisma:

```
routes → controllers → services → repositories → prisma → MySQL
```

```
// src/repositories/libros-repository.js
import { prisma } from '../db/prisma.js';

export function obtenerLibroPublicoPorSlug(slug) {
  return prisma.libro.findFirst({
    where: { slug, activo: true },
    select: { titulo: true, autor: true, precio: true, stock: true, sinopsis: true },
  });
}
```

Para las transacciones, los repositorios reciben `tx` como parámetro opcional (`db = prisma`), así el servicio puede encadenar varias operaciones dentro de un mismo `$transaction`.

## 13. Tests contra la base de prueba

Los tests de la API (Vitest + Supertest) usan la base `entrelineas_test`. Antes de cada archivo de tests, `test/setup.js` reemplaza `DATABASE_URL` por `TEST_DATABASE_URL`. Como `src/db/prisma.js` lee esa variable recién cuando se carga, todo el código (repositorios incluidos) queda apuntando a la base de prueba sin cambiar una línea. En la Fase 1, cuando haya tablas, se suma un script que aplica las migraciones a esa base antes de correr los tests.

## 14. Datos de ejemplo (seed)

El **seed** es un script que llena la base con datos de prueba: los 2 usuarios, los 8 géneros y los 60 libros de la Fase 1. Se registra en `prisma7.config.ts` (`migrations.seed`) y se ejecuta con `npx prisma db seed`.

Desde Prisma 7, `migrate dev` y `migrate reset` ya no lo corren solos: después de rehacer la base hay que ejecutarlo a mano. Usa `upsert` (crear o actualizar), así se puede correr varias veces sin duplicar datos.

## 15. Prisma Studio

```
npx prisma studio
```

Abre en el navegador una tabla editable de cada modelo. Sirve para revisar qué guardó un endpoint o cargar un dato a mano en desarrollo. No se usa contra la base de la demo.

## 16. Resumen de comandos

| Comando | Para qué |
|---|---|
| `npm run db:check` | Probar la conexión a MySQL |
| `npm run db:migrate -- --name <nombre>` | Crear y aplicar una migración ( `prisma migrate dev` ) |
| `npm run db:generate` | Regenerar el cliente ( `prisma generate` ) |
| `npm run db:studio` | Abrir Prisma Studio |
| `npx prisma migrate deploy` | Aplicar migraciones en la demo |
| `npx prisma migrate reset` | Rehacer la base de desarrollo desde cero (borra todo) |
| `npx prisma format` | Ordenar y validar `schema.prisma` |

## 17. Glosario

| Término | Definición |
|---|---|
| **ORM** | Herramienta que traduce entre tablas de la base y objetos del lenguaje |
| **Esquema** | Descripción de todas las tablas, columnas y relaciones |
| **Modelo** | Una tabla vista desde Prisma ( `Libro` ↔ `libros` ) |
| **Migración** | Archivo SQL versionado que cambia la estructura de la base |
| **Cliente generado** | Código que Prisma escribe a partir del esquema para consultar la base |
| **Driver adapter** | Conector de JavaScript que habla con MySQL en nombre de Prisma |
| **Pool de conexiones** | Grupo de conexiones abiertas que se reutilizan entre pedidos |
| **Clave foránea** | Columna que apunta al `id` de otra tabla ( `venta_id` ) |
| **Transacción** | Grupo de operaciones que se aplican todas o ninguna |
| **Rollback** | Deshacer todo lo hecho dentro de una transacción que falló |

| Término | Definición |
| --- | --- |
| **Idempotente** | Operación que da el mismo resultado si se ejecuta una o varias veces |
| **Seed** | Script que carga datos de ejemplo |
| **Base sombra** | Base temporal que usa `migrate dev` para comparar estados |
| **Mínimo privilegio** | Darle a cada usuario solo los permisos que necesita |
| **Type stripping** | Función de Node 24 que ejecuta TypeScript quitando los tipos al cargar el archivo |